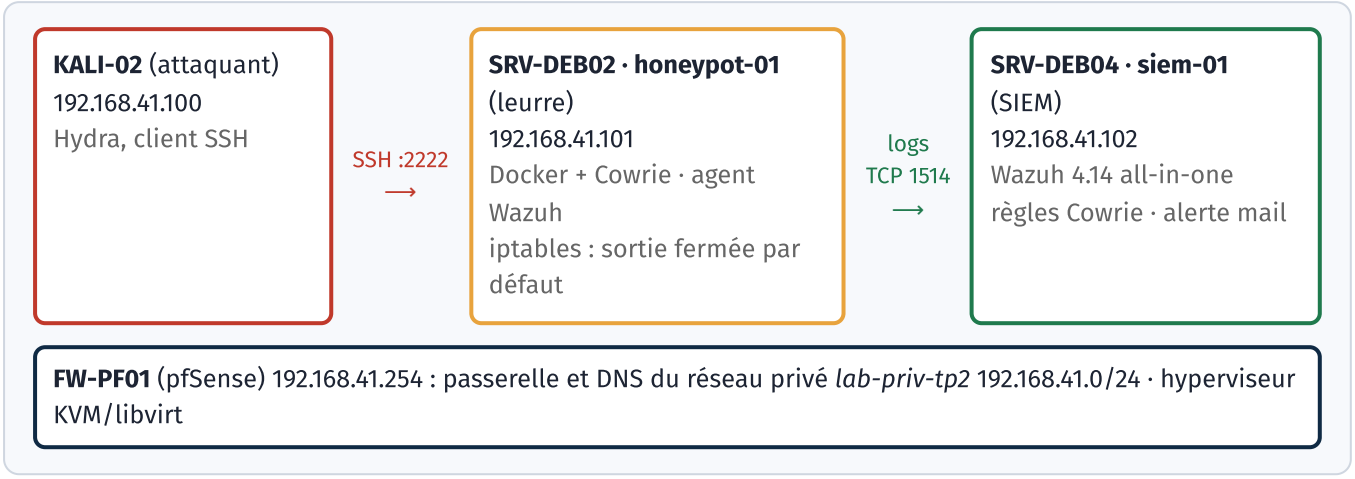
BTS SIO · BLOC 3 · CYBERSÉCURITÉ

Opération Honeypot

Fiche procédure — déployer, confiner et écouter un honeypot SSH (Cowrie + Wazuh)

Binôme	Thomas HIRON (seul)
Date	29/10/2026
Heure de début	9h30
Heure de fin	12h05
Poste	PC Pop!_OS 24.04 (i7-8665U, 31 Go) · KVM/libvirt · Lab SISR Manager

Architecture du laboratoire



Brique	Outil retenu	Rôle
Capteur	Cowrie (Docker)	faux serveur SSH, enregistre identifiants et commandes
Transport	agent Wazuh	envoie `cowrie.json` au SIEM, seule sortie autorisée
SIEM	Wazuh (indexer, manager, dashboard)	stockage, règles, tableau de bord
Alerte	mail (Postfix local)	connexion réussie sur le leurre = niveau 12

Sommaire

1. Préparation du laboratoire
2. Partie 1 — Déployer le capteur
3. Partie 2 — Lire les journaux
4. Partie 3 — Confiner le leurre
5. Partie 4 — Centraliser et alerter
6. Partie 5 — Analyser et conclure
7. Synthèse

Préparation du laboratoire

Mise en place des VM, du SIEM et des outils avant le TP.

0.1a — Inventaire des VM du laboratoire

Le labo tourne sur KVM/libvirt. Les quatre VM du TP sont sur le même réseau privé **192.168.41.0/24** (réseau libvirt *lab-priv-tp2*) :

- **FW-PF01** (192.168.41.254) : pfSense, passerelle et pare-feu
- **SRV-DEB02 / honeypot-01** (192.168.41.101) : le leurre (Docker + Cowrie)
- **SRV-DEB04 / siem-01** (192.168.41.102) : le SIEM Wazuh
- **KALI-02** (192.168.41.100) : le poste client « attaquant »

thomas@pop-os

```
thomas@pop-os:~$ virsh list
 Id   Name            State
------------------------------
 11   lab-srv-deb02   running
 13   lab-fw-pf01     running
 14   lab-kali-02     running
 15   lab-srv-deb04   running
```

Capturé le 29/10/2026 10:17:22

0.1b — Dimensionnement du SIEM (4 vCPU / 6 Go)

Wazuh « all-in-one » (manager + indexer + dashboard) demande au minimum 4 Go de RAM. La VM SIEM a été éteinte puis passée de 1 vCPU / 1 Go à **4 vCPU / 6 Go** :
`virsh setmaxmem/setmem lab-srv-deb04 6G --config` puis `virsh setvcpus lab-srv-deb04 4 --maximum --config` / `--config`.

thomas@pop-os

```
thomas@pop-os:~$ virsh dominfo lab-srv-deb04
Name:           lab-srv-deb04
State:          running
CPU(s):         4
Max memory:     6291456 KiB
Used memory:    6291456 KiB
```

Capturé le 29/10/2026 10:17:23

0.2a — Installation de Wazuh all-in-one (en cours)

Le script officiel installe sur la même VM les trois briques du SIEM : **Wazuh indexer** (stockage / indexation), **Wazuh manager** (analyse et règles) et **Wazuh dashboard** (interface web). L'URL générique `/4.x/` renvoyant une erreur 403, on utilise l'URL versionnée :
`curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh`

sisr@siem-01

```
sisr@siem-01:~$ sudo bash wazuh-install.sh -a
29/10/2026 10:13:40 INFO: Wazuh indexer cluster initialized.
29/10/2026 10:13:40 INFO: --- Wazuh server ---
29/10/2026 10:13:40 INFO: Starting the Wazuh manager installation.
29/10/2026 10:14:58 INFO: Wazuh manager installation finished.
29/10/2026 10:14:58 INFO: Wazuh manager vulnerability detection configuration finished.
29/10/2026 10:14:58 INFO: Starting service wazuh-manager.
29/10/2026 10:15:13 INFO: wazuh-manager service started.
29/10/2026 10:15:13 INFO: Starting Filebeat installation.
29/10/2026 10:15:23 INFO: Filebeat installation finished.
29/10/2026 10:15:24 INFO: Filebeat post-install configuration finished.
29/10/2026 10:15:24 INFO: Starting service filebeat.
29/10/2026 10:15:26 INFO: filebeat service started.
29/10/2026 10:15:26 INFO: --- Wazuh dashboard ---
29/10/2026 10:15:26 INFO: Starting Wazuh dashboard installation.
29/10/2026 10:16:40 INFO: Wazuh dashboard installation finished.
29/10/2026 10:16:40 INFO: Wazuh dashboard post-install configuration finished.
29/10/2026 10:16:40 INFO: Starting service wazuh-dashboard.
29/10/2026 10:16:41 INFO: wazuh-dashboard service started.
29/10/2026 10:16:42 INFO: Updating the internal users.
29/10/2026 10:16:49 INFO: A backup of the internal users has been saved in the /etc/wazuh-indexer/internal
users-backup folder.
29/10/2026 10:17:00 INFO: The filebeat.yml file has been updated to use the Filebeat Keystore username and
password.
29/10/2026 10:17:36 INFO: Initializing Wazuh dashboard web application.
```

Capturé le 29/10/2026 10:19:08

0.2b — Fin de l'installation de Wazuh

Les quatre services sont actifs. Les identifiants générés (compte **admin** du tableau de bord) sont conservés dans l'archive `wazuh-install-files.tar` (fichier `wazuh-passwords.txt`) : ils ne figurent pas dans ce document.

sisr@siem-01

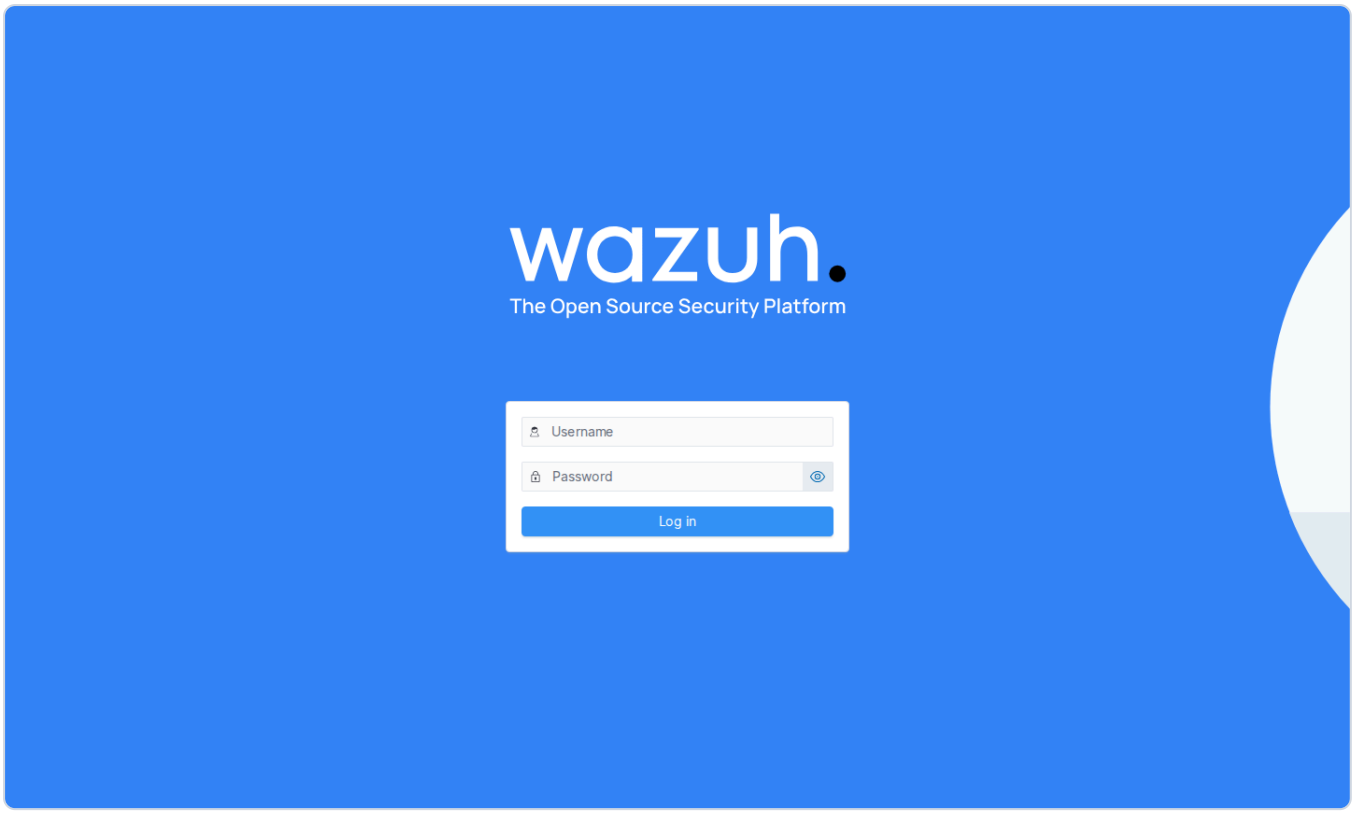
```
sisr@siem-01:~$ sudo tail /var/log/wazuh-install.log ; systemctl is-active wazuh-indexer wazuh-manager waz
uh-dashboard filebeat
29/10/2026 10:17:36 INFO: Initializing Wazuh dashboard web application.
29/10/2026 10:20:14 INFO: Wazuh dashboard web application initialized.
29/10/2026 10:20:14 INFO: --- Dependencies ----
29/10/2026 10:20:14 INFO: Removing gawk.
29/10/2026 10:20:22 INFO: Removing lsof.
29/10/2026 10:20:29 INFO: Installation finished.

active active active active
wazuh-install-files/wazuh-passwords.txt
```

Capturé le 29/10/2026 10:24:16

0.2c — Page de connexion du tableau de bord Wazuh

Le tableau de bord répond en HTTPS sur **https://192.168.41.102** (certificat auto-signé du labo).



Source : https://192.168.41.102/

Capturé le 29/10/2026 10:24:25

0.3a — Préparation du leurre : Docker, jq et image Cowrie

Sur SRV-DEB02 renommée **honeypot-01** :

- `sudo apt install -y docker.io jq` : Docker pour isoler Cowrie dans un conteneur, jq pour lire les logs JSON
- `sudo usermod -aG docker $USER` : utilisation de Docker sans sudo (reconnexion nécessaire)
- `docker pull cowrie/cowrie:latest` : l'image est téléchargée **avant** le confinement
- `mkdir ~/cowrie-logs && sudo chown 999:999 ~/cowrie-logs` : Cowrie tourne sous l'UID 999 dans le conteneur, il doit pouvoir écrire ses logs dans ce dossier monté

```
sisr@honeypot-01
sisr@honeypot-01:~$ hostnamectl ; docker --version ; docker images ; id ; ls -ldn ~/cowrie-logs
# docker --version ; jq --version
Docker version 20.10.24+dfsg1, build 297e128
jq-1.6

# docker images
REPOSITORY      TAG       IMAGE ID       CREATED      SIZE
cowrie/cowrie   latest    e99757182d5d   5 days ago   278MB

# id ; ls -ldn ~/cowrie-logs
uid=1000(sisr) gid=1000(sisr) groupes=1000(sisr),27(sudo),109(docker)
drwxr-xr-x 2 999 999 4096 29 oct.  10:23 /home/sisr/cowrie-logs
```

Capturé le 29/10/2026 10:24:35

0.4a — Installation de l'agent Wazuh sur le leurre

L'agent Wazuh assure le **transport** des logs du leurre vers le SIEM (TCP 1514, enrôlement sur TCP 1515). Il est installé **avant** le confinement, tant que le leurre a encore accès aux dépôts :

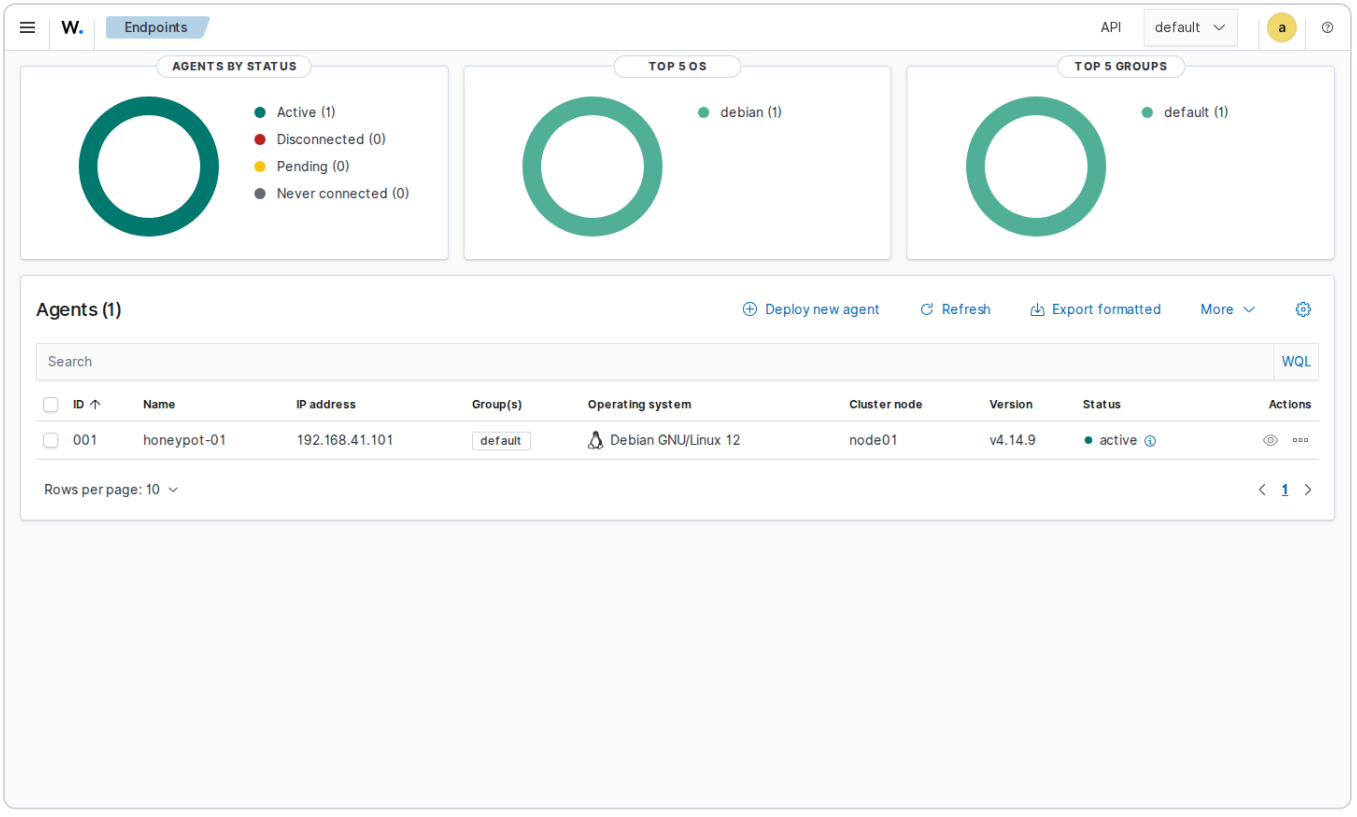
- `sudo apt install -y gnupg` (absent de l'image Debian cloud), puis import de la clé GPG Wazuh et ajout du dépôt `packages.wazuh.com/4.x/apt`
- `sudo WAZUH_MANAGER="192.168.41.102" WAZUH_AGENT_NAME="honeypot-01" apt install -y wazuh-agent=4.14.9-1` (même version que le serveur)
- `sudo systemctl enable --now wazuh-agent`

```
sisr@honeypot-01
sisr@honeypot-01:~$ dpkg -l wazuh-agent ; systemctl is-active wazuh-agent ; grep 'Connected' /var/ossec/logs/ossec.log
wazuh-agent 4.14.9-1
active
2026/10/29 10:35:37 wazuh-agentd: INFO: (4102): Connected to the server ([192.168.41.102]:1514/tcp).
```

Capturé le 29/10/2026 10:36:54

0.4b — L'agent honeypot-01 est actif dans le tableau de bord Wazuh

Dans le tableau de bord (menu **Agents management → Summary**), l'agent **honeypot-01** apparaît avec le statut **active** : le leurre envoie ses événements au SIEM.



Source : https://192.168.41.102/app/endpoints-summary#/agents-preview/
Capturé le 29/10/2026 10:36:37

0.5a — Incident : disque du SIEM saturé, agrandi à 60 Go

Incident rencontré : juste après l'installation, le disque de 20 Go du SIEM s'est retrouvé plein à 100 % : le module de détection de vulnérabilités télécharge et décompresse sa base CVE (~14 Go). Conséquence : l'indexer passe ses index en lecture seule et l'agent reste « Unknown ».
Correction :

- agrandissement à chaud du disque virtuel : `virsh blockresize lab-srv-deb04 vda 60G`
- dans la VM : `sudo apt-get clean` (pour libérer un peu de place), `sudo growpart /dev/vda 1`, `sudo resize2fs /dev/vda1`
- redémarrage de `wazuh-indexer`, `wazuh-manager`, `wazuh-dashboard`, puis levée du verrou `index.blocks.read_only_allow_delete` sur les index `wazuh-*`

Leçon : respecter le dimensionnement recommandé par l'éditeur (50 Go minimum pour Wazuh all-in-one).

sisr@siem-01

```
sisr@siem-01:~$ df -h / ; du -sh /var/ossec/tmp /var/ossec/queue ; agent_control -l
Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur
/dev/vda1           59G     16G   42G  27% /

9,7G    /var/ossec/tmp
1,5G    /var/ossec/queue

   ID: 000, Name: siem-01 (server), IP: 127.0.0.1, Active/Local
   ID: 001, Name: honeypot-01, IP: any, Active
```

Capturé le 29/10/2026 10:36:55

Partie 1 — Déployer le capteur

Cowrie simule un serveur SSH : il accepte les connexions et enregistre identifiants et commandes, sans jamais donner un vrai accès.

1.1a — Lancement du conteneur Cowrie

Sur honeypot-01, depuis le dossier personnel :

```
docker run -d --name cowrie --restart unless-stopped -p 2222:2222 -v $PWD/cowrie-logs:/cowrie/cowrie-git/var/log/cowrie cowrie/cowrie:latest
```

- `-d` : en arrière-plan ; `--restart unless-stopped` : redémarre avec la VM
- `-p 2222:2222` : le faux SSH écoute sur le port **2222** de la VM
- `-v` : les journaux sont écrits sur la VM (`~/cowrie-logs`) et survivent au conteneur

Attendu : le conteneur est **Up** et le port 2222 est en écoute.

sisr@honeypot-01

```
sisr@honeypot-01:~$ docker ps ; ss -ltnp | grep 2222 ; ls -l ~/cowrie-logs
NAMES     IMAGE                  STATUS              PORTS
cowrie    cowrie/cowrie:latest   Up About a minute   0.0.0.0:2222->2222/tcp, :::2222->2222/tcp, 2223/tcp

State  Recv-Q Send-Q Local Address:Port  Peer Address:PortProcess
LISTEN 0      4096         0.0.0.0:2222       0.0.0.0:*    users:(("docker-proxy",pid=6876,fd=4))
LISTEN 0      4096            [::]:2222          [::]:*    users:(("docker-proxy",pid=6881,fd=4))

total 0
-rw-rw-r-- 1 999 systemd-journal 0 29 oct.  11:11 cowrie.json
```

Capturé le 29/10/2026 11:12:37

1.2a — Le faux shell vu par l'attaquant (session rejouée)

Depuis **KALI-02** (192.168.41.100) : `ssh -p 2222 root@192.168.41.101`. Le mot de passe saisi est **accepté** et on obtient un shell. Cowrie enregistre chaque session terminal (ttylog) : ci-dessous la session de l'attaquant **rejouée** avec l'outil `playlog` de Cowrie, exactement telle qu'elle s'est affichée côté Kali. Indices que ce shell est **simulé** : le nom de machine affiché est `svr04` (et non honeypot-01), le `/etc/passwd` contient un utilisateur fictif `phil`, et `ls` montre un répertoire vide. Rien de ce qui est tapé ne touche une vraie machine.

sisr@honeypot-01

```
sisr@honeypot-01:~$ playlog var/lib/cowrie/tty/20261029-101517-6a9afc975e78-0i.log
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@svr04:~# whoami
root
root@svr04:~# ls
root@svr04:~# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
sshd:x:101:65534::/var/run/sshd:/usr/sbin/nologin
phil:x:1000:1000:Phil California,,,:/home/phil:/bin/bash
root@svr04:~# uname -a
Linux svr04 6.1.0-21-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.90-1 (2024-05-03) x86_64 GNU/Linux
root@svr04:~#
```

Capturé le 29/10/2026 11:17:52

1.2b — Côté défenseur : la connexion et les commandes sont journalisées

Dans `~/cowrie-logs/cowrie.json`, Cowrie a enregistré l'IP source (Kali), l'identifiant, le **mot de passe en clair** et chaque commande tapée (heures en UTC). Le mot de passe réel est masqué dans ce document.

sisr@honeypot-01

```
sisr@honeypot-01:~$ jq -c '{heure,evenement,ip,utilisateur,mdp,commande}' ~/cowrie-logs/cowrie.json
{"heure":"10:15:17","evenement":"cowrie.login.success","ip":"192.168.41.100","utilisateur":"root","mdp":"•
•••••••"}
{"heure":"10:15:55","evenement":"cowrie.command.input","ip":"192.168.41.100","commande":"whoami"}
{"heure":"10:16:04","evenement":"cowrie.command.input","ip":"192.168.41.100","commande":"ls"}
{"heure":"10:16:14","evenement":"cowrie.command.input","ip":"192.168.41.100","commande":"cat /etc/passwd"}
{"heure":"10:16:28","evenement":"cowrie.command.input","ip":"192.168.41.100","commande":"uname -a"}
```

Capturé le 29/10/2026 11:17:53

1.2c — Test des mots de passe : root refusé, un autre accepté

Deuxième connexion depuis Kali : le mot de passe `root` est **refusé** (`cowrie.login.failed`), puis le mot de passe suivant est **accepté** (`cowrie.login.success`). On remarque qu'une faute de frappe (`admin1223` au lieu de `admin123`) est acceptée elle aussi : Cowrie accepte n'importe quel mot de passe pour root, sauf quelques-uns (`root`, `123456`, tout ce qui contient « honeypot »).

sisr@honeypot-01

```
sisr@honeypot-01:~$ jq -c 'select(.eventid|test("login")) | {heure,evenement,ip,utilisateur,mdp}' ~/cowrie-logs/cowrie.json
{"heure":"10:15:17","evenement":"cowrie.login.success","ip":"192.168.41.100","utilisateur":"root","mdp":"••••••••"}
{"heure":"10:20:21","evenement":"cowrie.login.failed","ip":"192.168.41.100","utilisateur":"root","mdp":"root"}
{"heure":"10:20:32","evenement":"cowrie.login.success","ip":"192.168.41.100","utilisateur":"root","mdp":"admin1223"}
```

Capturé le 29/10/2026 11:21:15

Réponses aux questions

Le mot de passe a-t-il été « accepté » ? Qu'en déduisez-vous ?

`root` a été refusé, mais tous les autres mots de passe essayés ont été acceptés, y compris une faute de frappe (`admin1223`). La configuration par défaut de Cowrie (`userdb`) accepte presque tout pour root, et ne refuse que quelques mots de passe (`root`, `123456`, tout ce qui contient « honeypot »).

- Le but est de **laisser entrer l'attaquant** pour observer ce qu'il fait après la connexion.
- Refuser quelques mots de passe rend le leurre **plus crédible** : un serveur qui accepte absolument tout se repère vite.
- Ce que voit l'attaquant est un **système simulé** (machine `svr04`, faux utilisateur `phil`, système de fichiers fictif). Il croit avoir compromis un serveur, alors qu'aucune commande n'est réellement exécutée et que tout est enregistré : IP, identifiants, mots de passe en clair, commandes.
- Conséquence côté défense : un mot de passe « accepté » sur un honeypot ne prouve rien. Et il ne faut jamais y taper un vrai mot de passe, puisqu'il est stocké en clair.

Partie 2 — Lire les journaux

Chaque événement est enregistré en JSON, une ligne par événement.

2.1a — Côté attaquant (Kali) : les connexions de test

Depuis Kali, plusieurs connexions avec des identifiants différents pour alimenter les journaux :

- `ssh -p 2222 admin@192.168.41.101`, mot de passe `admin`
- `ssh -p 2222 root@192.168.41.101`, mot de passe `123456`
- `ssh -p 2222 ubuntu@192.168.41.101`, mot de passe `ubuntu`

(l'historique zsh n'est écrit qu'à la fermeture du terminal : seules les premières commandes y figurent)

kali@kali-02

```
kali@kali-02:~$ history ; ssh-keygen -F '[192.168.41.101]:2222' -l
ssh -p 2222 root@192.168.41.101
exit

# empreinte de la clé SSH du leurre mémorisée par Kali :
[192.168.41.101]:2222 ED25519 SHA256:14pxUMwqLCv26ioMxPdXSYkTftv/tODuRb1llY5JZNw
```

Capturé le 29/10/2026 11:25:55

2.1b — Le flux brut : un événement JSON par ligne

`tail -f` affiche les événements au fil de l'eau. Chaque ligne est un objet JSON indépendant (format **NDJSON**), avec toujours `eventid`, `timestamp`, `src_ip` et `session`. On repère les trois types demandés : `cowrie.login.failed`, `cowrie.login.success` et `cowrie.command.input`.

sisr@honeypot-01

```
sisr@honeypot-01:~$ tail -f ~/cowrie-logs/cowrie.json
{"session":"7156f46c2645","protocol":"ssh","src_ip":"192.168.41.100","src_port":34618,"dst_ip":"172.17.0.
2","dst_port":2222,"input":"exit","eventid":"cowrie.command.input","sensor":"1fb733f121c8","uuid":"5458743
a-d0ea-11f1-9b92-2aedfafa4905","timestamp":"2026-10-29T10:20:42.565196Z","message":"CMD: exit"}
{"session":"7fefcd9b647b","protocol":"ssh","src_ip":"192.168.41.100","src_port":48712,"dst_ip":"172.17.0.
2","dst_port":2222,"eventid":"cowrie.session.connect","sensor":"1fb733f121c8","uuid":"5458743a-d0ea-11f1-9
b92-2aedfafa4905","timestamp":"2026-10-29T10:23:38.703545Z","message":"New connection: 192.168.41.100:4871
2 (172.17.0.2:2222) [session: 7fefcd9b647b]"}
{"session":"7fefcd9b647b","protocol":"ssh","src_ip":"192.168.41.100","src_port":48712,"dst_ip":"172.17.0.
2","dst_port":2222,"username":"admin","password":"admin","eventid":"cowrie.login.success","sensor":"1fb733
f121c8","uuid":"5458743a-d0ea-11f1-9b92-2aedfafa4905","timestamp":"2026-10-29T10:23:46.636295Z","messag
e":"login attempt [admin/admin] succeeded"}
{"session":"7fefcd9b647b","protocol":"ssh","src_ip":"192.168.41.100","src_port":48712,"dst_ip":"172.17.0.
2","dst_port":2222,"input":"exit","eventid":"cowrie.command.input","sensor":"1fb733f121c8","uuid":"5458743
a-d0ea-11f1-9b92-2aedfafa4905","timestamp":"2026-10-29T10:23:54.970748Z","message":"CMD: exit"}
{"session":"9bdf303d5050","protocol":"ssh","src_ip":"192.168.41.100","src_port":58988,"dst_ip":"172.17.0.
2","dst_port":2222,"eventid":"cowrie.session.connect","sensor":"1fb733f121c8","uuid":"5458743a-d0ea-11f1-9
b92-2aedfafa4905","timestamp":"2026-10-29T10:24:11.022024Z","message":"New connection: 192.168.41.100:5898
8 (172.17.0.2:2222) [session: 9bdf303d5050]"}
{"session":"9bdf303d5050","protocol":"ssh","src_ip":"192.168.41.100","src_port":58988,"dst_ip":"172.17.0.
2","dst_port":2222,"username":"root","password":"123456","eventid":"cowrie.login.failed","sensor":"1fb733f
121c8","uuid":"5458743a-d0ea-11f1-9b92-2aedfafa4905","timestamp":"2026-10-29T10:24:21.875942Z","messag
e":"login attempt [root/123456] failed"}
{"session":"dea00b9ce65a","protocol":"ssh","src_ip":"192.168.41.100","src_port":37580,"dst_ip":"172.17.0.
2","dst_port":2222,"eventid":"cowrie.session.connect","sensor":"1fb733f121c8","uuid":"5458743a-d0ea-11f1-9
b92-2aedfafa4905","timestamp":"2026-10-29T10:24:41.180193Z","message":"New connection: 192.168.41.100:3758
0 (172.17.0.2:2222) [session: dea00b9ce65a]"}
{"session":"dea00b9ce65a","protocol":"ssh","src_ip":"192.168.41.100","src_port":37580,"dst_ip":"172.17.0.
2","dst_port":2222,"username":"ubuntu","password":"ubuntu","eventid":"cowrie.login.success","sensor":"1fb7
33f121c8","uuid":"5458743a-d0ea-11f1-9b92-2aedfafa4905","timestamp":"2026-10-29T10:24:47.498481Z","messag
e":"login attempt [ubuntu/ubuntu] succeeded"}
{"session":"dea00b9ce65a","protocol":"ssh","src_ip":"192.168.41.100","src_port":37580,"dst_ip":"172.17.0.
2","dst_port":2222,"input":"exit","eventid":"cowrie.command.input","sensor":"1fb733f121c8","uuid":"5458743
a-d0ea-11f1-9b92-2aedfafa4905","timestamp":"2026-10-29T10:25:00.633919Z","message":"CMD: exit"}
```

Capturé le 29/10/2026 11:26:05

2.1c — Anatomie d'un événement cowrie.login.failed

Le dernier échec de connexion, mis en forme par `jq` : chaque champ se lit facilement.

sisr@honeypot-01

```
sisr@honeypot-01:~$ jq -s '[.[] | select(.eventid=="cowrie.login.failed")] | last' ~/cowrie-logs/cowrie.js
on
{
  "session": "9bdf303d5050",
  "protocol": "ssh",
  "src_ip": "192.168.41.100",
  "src_port": 58988,
  "dst_ip": "172.17.0.2",
  "dst_port": 2222,
  "username": "root",
  "password": "123456",
  "eventid": "cowrie.login.failed",
  "sensor": "1fb733f121c8",
  "uuid": "5458743a-d0ea-11f1-9b92-2aedfafa4905",
  "timestamp": "2026-10-29T10:24:21.875942Z",
  "message": "login attempt [root/123456] failed"
}
```

Capturé le 29/10/2026 11:26:04

2.2a — Mots de passe les plus tentés (échecs)

Chaîne de traitement : `jq` garde les échecs et extrait le mot de passe, `sort | uniq -c` regroupe et compte, `sort -rn` trie du plus fréquent au moins fréquent, `head` garde le top 10. Peu de lignes : Cowrie accepte presque tous les mots de passe, les échecs sont donc rares.

sisr@honeypot-01

```
sisr@honeypot-01:~$ jq -r 'select(.eventid=="cowrie.login.failed") | .password' ~/cowrie-logs/cowrie.json
| sort | uniq -c | sort -rn | head
      1 root
      1 123456
```

Capturé le 29/10/2026 11:27:44

2.2b — Toutes les tentatives : couples identifiant / mot de passe

Même logique sur toutes les tentatives (réussies et échouées) : c'est ce tableau qui servira de point de comparaison avec du trafic réel.

sisr@honeypot-01

```
sisr@honeypot-01:~$ jq -r 'select(.eventid|test("cowrie.login")) | "\(.eventid) \(.username) \(.passwor
d)"' ~/cowrie-logs/cowrie.json | sort | uniq -c | sort -rn
      1 success ubuntu ubuntu
      1 success root ••••••••
      1 success root admin1223
      1 success admin admin
      1 failed root root
      1 failed root 123456
```

Capturé le 29/10/2026 11:27:45

Réponses aux questions

À quoi sert chaque champ d'un événement cowrie.login.failed ?

- `timestamp` : date et heure précises de la tentative, en UTC (le Z final). Il sert à reconstituer la chronologie et à mesurer le rythme des tentatives (indice robot / humain).
- `src_ip` : adresse IP de l'attaquant. C'est elle qu'on enrichit (géolocalisation, réputation AbuseIPDB) et qu'on bloque côté pare-feu.
- `username` : identifiant essayé. Il révèle les comptes visés (root, admin, comptes de service).
- `password` : mot de passe essayé, **en clair**. Il montre les dictionnaires utilisés et les mots de passe à interdire chez soi.

Autres champs utiles : `session` (relie tous les événements d'une même connexion), `src_port` / `dst_port`, `eventid` (type d'événement).

Tableau de nos essais

Identifiant tenté	Mot de passe	Nombre	Résultat
root	(vrai mot de passe du lab, masqué)	1	accepté
root	root	1	refusé
root	admin1223	1	accepté
admin	admin	1	accepté
root	123456	1	refusé
ubuntu	ubuntu	1	accepté

Partie 3 — Confiner le leurre

Un honeypot qui peut initier des connexions vers l'extérieur devient une arme pour l'attaquant : on ferme la sortie, et on le prouve.

3.0a — Avant confinement : le leurre et le conteneur sortent librement

État **avant** le confinement : la VM comme le conteneur Cowrie peuvent joindre Internet. Un attaquant qui s'échapperait du faux shell (ou une commande `wget` / `curl` que Cowrie exécute réellement pour récupérer un fichier) pourrait donc télécharger des outils ou attaquer d'autres machines depuis notre réseau.

sisr@honeypot-01

```
sisr@honeypot-01:~$ curl -sI -m 5 https://example.com ; docker exec cowrie python3 -c 'urlopen("https://example.com")'
# depuis la VM :
HTTP/2 200
# depuis le conteneur Cowrie :
HTTP 200
```

Capturé le 29/10/2026 11:27:45

3.1a — Le script de confinement

Le leurre et les autres VM sont sur le même réseau (192.168.41.0/24) : pfSense ne voit pas le trafic interne. Le confinement est donc posé **sur le leurre lui-même**, avec `iptables`, à deux niveaux :

- chaîne **OUTPUT** : ce que la VM elle-même émet ;
- chaîne **DOCKER-USER** : ce que le conteneur Cowrie émet (son trafic passe par FORWARD, pas par OUTPUT : sans cette règle, le conteneur sortirait quand même).

Politique : **refus par défaut**, seules exceptions le DNS (vers pfSense) et l'envoi des logs (agent Wazuh vers le SIEM, TCP 1514-1515). Chaque paquet refusé est journalisé.

sisr@honeypot-01

```
sisr@honeypot-01:~$ cat ~/confinement.sh
#!/bin/bash
# Confinement du leurre : refus par défaut en sortie, sauf DNS et envoi des logs
SIEM=192.168.41.102
DNS=192.168.41.254

# 1) Sortie de la VM (chaîne OUTPUT)
iptables -F OUTPUT
iptables -A OUTPUT -o lo -j ACCEPT                                      # boucle locale
iptables -A OUTPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT     # réponses aux connexions entr
antes (SSH admin, Cowrie)
iptables -A OUTPUT -p udp -d $DNS --dport 53 -j ACCEPT                  # DNS
iptables -A OUTPUT -p tcp -d $DNS --dport 53 -j ACCEPT                  # DNS (TCP)
iptables -A OUTPUT -p tcp -d $SIEM --dport 1514:1515 -j ACCEPT          # logs vers le SIEM (agent Waz
uh)
iptables -A OUTPUT -j LOG --log-prefix "LEURRE-SORTIE-BLOQUEE: "
iptables -P OUTPUT DROP                                                 # tout le reste : refusé

# 2) Sortie du conteneur Cowrie (chaîne DOCKER-USER)
iptables -F DOCKER-USER
iptables -A DOCKER-USER -i docker0 -m conntrack --ctstate NEW -j LOG --log-prefix "COWRIE-SORTIE-BLOQUEE:
"
iptables -A DOCKER-USER -i docker0 -m conntrack --ctstate NEW -j DROP      # le conteneur n'initie rien
iptables -A DOCKER-USER -j RETURN
```

Capturé le 29/10/2026 11:31:17

3.1b — Règles appliquées

Après `sudo ~/confinement.sh` : politique **DROP** sur OUTPUT, compteurs de paquets visibles pour chaque règle.

sisr@honeypot-01

```
sisr@honeypot-01:~$ sudo ~/confinement.sh ; sudo iptables -L OUTPUT -n -v --line-numbers ; sudo iptables -
L DOCKER-USER -n -v --line-numbers
Chain OUTPUT (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     0    --  *      lo      0.0.0.0/0            0.0.0.0/0
2      514  221K ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           ctstate RELA
TED,ESTABLISHED
3        0     0 ACCEPT     17   --  *      *       0.0.0.0/0            192.168.41.254      udp dpt:53
4        0     0 ACCEPT     6    --  *      *       0.0.0.0/0            192.168.41.254      tcp dpt:53
5        0     0 ACCEPT     6    --  *      *       0.0.0.0/0            192.168.41.102      tcp dpts:151
4:1515
6        0     0 LOG        0    --  *      *       0.0.0.0/0            0.0.0.0/0           LOG flags 0
level 4 prefix "LEURRE-SORTIE-BLOQUEE: "

Chain DOCKER-USER (1 references)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 LOG        0    --  docker0 *       0.0.0.0/0            0.0.0.0/0            ctstate NEW
LOG flags 0 level 4 prefix "COWRIE-SORTIE-BLOQUEE: "
2        0     0 DROP       0    --  docker0 *       0.0.0.0/0            0.0.0.0/0            ctstate NEW
3        0     0 RETURN     0    --  *      *       0.0.0.0/0            0.0.0.0/0
```

Capturé le 29/10/2026 11:31:17

3.2a — Preuve : la VM ne peut plus sortir vers Internet

Le test **doit échouer** : c'est la preuve du confinement. Le nom est bien résolu (DNS autorisé), mais la connexion HTTPS expire au bout de 5 s (code 28 = délai dépassé).

sisr@honeypot-01

```
sisr@honeypot-01:~$ curl -m 5 https://example.com
curl: (28) Connection timed out after 5001 milliseconds
code retour curl : 28
```

Capturé le 29/10/2026 11:33:21

3.2b — Preuve : le conteneur Cowrie ne peut plus sortir

Même test depuis l'intérieur du conteneur (l'image Cowrie n'a ni shell ni curl : on utilise son Python). Le conteneur ne peut même plus résoudre un nom : sa requête DNS est une nouvelle connexion sortante, refusée par la règle DOCKER-USER. Cowrie ne peut donc rien télécharger pour le compte d'un attaquant.

sisr@honeypot-01

```
sisr@honeypot-01:~$ docker exec cowrie python3 -c "urllib.request.urlopen('https://example.com', timeout=
5)"
urllib.error.URLError: <urlopen error [Errno -3] Temporary failure in name resolution>
```

Capturé le 29/10/2026 11:32:47

3.2c — Preuve : pas de rebond vers les autres machines du réseau

Le réseau du labo est « à plat » : sans confinement, un attaquant ayant pris le contrôle du leurre pourrait attaquer Kali, pfSense ou le SIEM (déplacement latéral). Les tentatives de connexion vers d'autres machines du réseau sont bloquées.

```
sisr@honeypot-01
sisr@honeypot-01:~$ timeout 5 bash -c 'echo > /dev/tcp/192.168.41.100/22' ; timeout 5 bash -c 'echo > /dev/tcp/192.168.41.254/443'
192.168.41.100:22 (Kali, SSH) -> BLOQUÉ
192.168.41.254:443 (pfSense, HTTPS) -> BLOQUÉ
```

Capturé le 29/10/2026 11:32:10

3.2d — Les exceptions fonctionnent : DNS et envoi des logs

Le confinement ne casse pas ce qui est nécessaire : la résolution DNS fonctionne et l'agent Wazuh reste connecté au SIEM (keepalive récent).

```
sisr@honeypot-01
sisr@honeypot-01:~$ getent ahostsv4 example.com ; systemctl is-active wazuh-agent ; grep status /var/ossec/var/run/wazuh-agentd.state
# DNS :
104.20.23.154
172.66.147.243
# agent Wazuh :
active
status='connected'
last_keepalive='2026-10-29 11:32:53'
```

Capturé le 29/10/2026 11:33:03

3.2e — Chaque tentative bloquée est journalisée

Les règles LOG écrivent chaque paquet refusé dans le journal du noyau, avec l'adresse et le port visés : utile pour savoir ce qu'un attaquant a tenté de joindre. Les compteurs de paquets des règles de refus ne sont plus à zéro.

sisr@honeypot-01

```
sisr@honeypot-01:~$ sudo journalctl -k | grep SORTIE-BLOQUEE ; iptables -L -n -v
2026-10-29T11:31:55+0100 honeypot-01 kernel: COWRIE-SORTIE-BLOQUEE: IN=docker0 OUT=ens3 PHYSIN=vethcf06568
SRC=172.17.0.2 DST=192.168.41.254 DF PROTO=UDP SPT=60526 DPT=53
2026-10-29T11:31:55+0100 honeypot-01 kernel: COWRIE-SORTIE-BLOQUEE: IN=docker0 OUT=ens3 PHYSIN=vethcf06568
SRC=172.17.0.2 DST=192.168.41.254 DF PROTO=UDP SPT=60526 DPT=53
2026-10-29T11:32:00+0100 honeypot-01 kernel: LEURRE-SORTIE-BLOQUEE: IN= OUT=ens3 SRC=192.168.41.101 DST=19
2.168.41.100 DF PROTO=TCP SPT=40038 DPT=22 SYN
2026-10-29T11:32:01+0100 honeypot-01 kernel: LEURRE-SORTIE-BLOQUEE: IN= OUT=ens3 SRC=192.168.41.101 DST=19
2.168.41.100 DF PROTO=TCP SPT=40038 DPT=22 SYN
2026-10-29T11:32:03+0100 honeypot-01 kernel: LEURRE-SORTIE-BLOQUEE: IN= OUT=ens3 SRC=192.168.41.101 DST=19
2.168.41.100 DF PROTO=TCP SPT=40038 DPT=22 SYN
2026-10-29T11:32:05+0100 honeypot-01 kernel: LEURRE-SORTIE-BLOQUEE: IN= OUT=ens3 SRC=192.168.41.101 DST=19
2.168.41.254 DF PROTO=TCP SPT=40314 DPT=443 SYN
2026-10-29T11:32:06+0100 honeypot-01 kernel: LEURRE-SORTIE-BLOQUEE: IN= OUT=ens3 SRC=192.168.41.101 DST=19
2.168.41.254 DF PROTO=TCP SPT=40314 DPT=443 SYN
2026-10-29T11:32:08+0100 honeypot-01 kernel: LEURRE-SORTIE-BLOQUEE: IN= OUT=ens3 SRC=192.168.41.101 DST=19
2.168.41.254 DF PROTO=TCP SPT=40314 DPT=443 SYN

Chain OUTPUT (policy DROP 10 packets, 600 bytes)
   10   600 LOG        0    --  *      *       0.0.0.0/0            0.0.0.0/0            LOG flags 0 level
4 prefix "LEURRE-SORTIE-BLOQUEE: "
Chain DOCKER-USER (1 references)
 pkts bytes target     prot opt in     out     source               destination
    8   456 DROP       0    --  docker0 *       0.0.0.0/0            0.0.0.0/0            ctstate NEW
```

Capturé le 29/10/2026 11:32:11

3.2f — Preuve : depuis le faux shell, wget échoue

Depuis Kali, dans le faux shell : `wget http://example.com/malware.sh`. Cowrie tente réellement le téléchargement (pour récupérer les outils des attaquants), mais la requête DNS du conteneur est bloquée par la règle DOCKER-USER : **rien n'est téléchargé**. Côté attaquant, la commande reste sans résultat.

sisr@honeypot-01

```
sisr@honeypot-01:~$ playlog <session de l'attaquant> ; docker logs cowrie | grep -E 'wget|DNS|blocked'
root@svr04:~# wget http://example.com/malware.sh
--2026-10-29 10:38:03--  http://example.com/malware.sh
Resolving example.com (example.com)... failed: Temporary failure in name resolution.
wget: unable to resolve host address ‘example.com’
root@svr04:~#
# ce que Cowrie a journalisé :
10:37:03 [ssh,ce3c682bdb19,192.168.41.100] CMD: wget http://example.com/malware.sh
10:38:03 [cowrie.core.network#info] DNS lookup failed for 'example.com': [Query(b'example.com', 1, 1)]
10:38:03 [cowrie.commands.wget.Command_wget#info] Attempt to access blocked network address
```

Capturé le 29/10/2026 11:39:46

Réponses aux questions

Quelles règles avez-vous posées, et pourquoi ces exceptions-là seulement ?

Politique de **refus par défaut** en sortie (`iptables -P OUTPUT DROP`) sur la VM, et refus de toute nouvelle connexion émise par le conteneur (chaîne DOCKER-USER). Les seules exceptions :

- les **réponses** aux connexions entrantes (ESTABLISHED,RELATED) : sans elles, le leurre ne pourrait pas répondre à l'attaquant (Cowrie) ni à l'administrateur (SSH 22) ;

- le **DNS** vers pfSense (UDP/TCP 53), le seul résolveur autorisé ;
- l'**envoi des logs** vers le SIEM uniquement (192.168.41.102, TCP 1514-1515 pour l'agent Wazuh). C'est la raison d'être du leurre : ce qu'il voit doit sortir, et rien d'autre.

Tout le reste est refusé et journalisé (LOG). Le principe est celui du **moindre privilège** : chaque exception ouverte est une porte possible pour l'attaquant, on ne garde donc que celles qui sont indispensables au fonctionnement du capteur.

Que risquerait l'établissement si ce confinement n'existait pas ?
Un attaquant qui sort du faux shell, ou qui exploite une faille de Cowrie ou de Docker, contrôlerait une machine **à l'intérieur du réseau de l'établissement**. Il pourrait :

- rebondir vers les autres machines (le réseau du labo est à plat : Kali, pfSense, le SIEM sont joignables), c'est-à-dire un déplacement latéral ;
- se servir du leurre pour attaquer des tiers sur Internet (scans, brute force, DDoS, envoi de spam, minage), avec **l'adresse IP de l'établissement**. Celui-ci en porterait la **responsabilité juridique** et verrait sa réputation atteinte (IP mise sur liste noire) ;
- télécharger et héberger des outils ou des fichiers illicites.

Le honeypot deviendrait alors une arme contre l'établissement, au lieu de le protéger.

Partie 4 — Centraliser et alerter

Le leurre peut tomber ; les logs, non. Ils partent vers un SIEM situé sur une autre machine, qui déclenche une alerte.

4.1a — L'agent lit désormais le journal de Cowrie

Choix technique : le sujet propose Filebeat vers Logstash (port 5044). Or le SIEM retenu est **Wazuh**, qui n'utilise pas Logstash : son transport natif est l'**agent Wazuh**, déjà installé sur le leurre. On lui ajoute simplement une source dans `/var/ossec/etc/ossec.conf` (`log_format json` : chaque ligne est découpée en champs), puis `sudo systemctl restart wazuh-agent`. L'envoi utilise la seule sortie autorisée par le confinement (TCP 1514 vers le SIEM), chiffrée par l'agent.

sisr@honeypot-01

```
sisr@honeypot-01:~$ tail -n 7 /var/ossec/etc/ossec.conf ; systemctl is-active wazuh-agent ; grep cowrie /v
ar/ossec/logs/ossec.log

<ossec_config>
  <localfile>
    <log_format>json</log_format>
    <location>/home/sisr/cowrie-logs/cowrie.json</location>
  </localfile>
</ossec_config>

active
2026/10/29 11:41:15 wazuh-logcollector: INFO: (1950): Analyzing file: '/home/sisr/cowrie-logs/cowrie.jso
n'.
```

Capturé le 29/10/2026 11:42:22

4.1b — Le SIEM sait décoder un événement Cowrie… mais n'a pas de règle adaptée

`wazuh-logtest` rejoue un événement dans le moteur d'analyse du SIEM, pour savoir ce qu'il en fera :

- **Phase 2 (décodage)** : le décodeur `json` découpe correctement tous les champs Cowrie.
- **Phase 3 (règles)** : aucune règle n'est prévue pour Cowrie. L'événement tombe sur une règle générique de niveau 2 (« Unknown problem »), en dessous du seuil d'alerte (3) : **rien n'apparaîtrait dans le tableau de bord**.

Il faut donc écrire nos propres règles (étape 4.2).

sisr@siem-01

```
sisr@siem-01:~$ echo '<ligne cowrie.login.failed>' | sudo /var/ossec/bin/wazuh-logtest
**Phase 2: Completed decoding.
        name: 'json'
        dst_port: '2222'
        eventid: 'cowrie.login.failed'
        message: 'login attempt [root/123456] failed'
        password: '123456'
        session: '9bdf303d5050'
        src_ip: '192.168.41.100'
        src_port: '58988'
        timestamp: '2026-10-29T10:24:21.875942Z'
        username: 'root'

**Phase 3: Completed filtering (rules).
        id: '1002'
        level: '2'
        description: 'Unknown problem somewhere in the system.'
        groups: '['syslog', 'errors']'
```

Capturé le 29/10/2026 11:42:38

4.2a — Les règles Cowrie sont en place sur le SIEM

Règles personnalisées ajoutées dans `/var/ossec/etc/rules/local_rules.xml` sur **siem-01**, puis `sudo systemctl restart wazuh-manager`. Une règle mère (100100) reconnaît tout événement dont le champ `eventid` commence par `cowrie.`, et trois règles filles graduent la gravité :

Règle	Événement	Niveau	Justification
100100	tout événement Cowrie	3	visible dans le tableau de bord
100101	`cowrie.login.failed`	5	bruit de fond normal d'un honeypot
100102	`cowrie.login.success`	**12**	sur un leurre, une connexion réussie est **toujours anormale** : c'est l'alerte critique
100103	`cowrie.command.input`	7	montre ce que fait l'attaquant

sisr@siem-01

```
sisr@siem-01:~$ sudo cat /var/ossec/etc/rules/local_rules.xml ; systemctl is-active wazuh-manager
<group name="cowrie,honeypot,">
  <rule id="100100" level="3">
    <decoded_as>json</decoded_as>
    <field name="eventid">^cowrie\.</field>
    <description>Cowrie : $(eventid) depuis $(src_ip)</description>
  </rule>

  <rule id="100101" level="5">
    <if_sid>100100</if_sid>
    <field name="eventid">^cowrie.login.failed$</field>
    <description>Honeypot : echec de connexion $(username)/$(password) depuis $(src_ip)</description>
    <group>authentication_failed,</group>
  </rule>

  <rule id="100102" level="12">
    <if_sid>100100</if_sid>
    <field name="eventid">^cowrie.login.success$</field>
    <description>Honeypot : connexion REUSSIE sur le leurre ($(username) depuis $(src_ip))</description>
    <group>authentication_success,</group>
  </rule>

  <rule id="100103" level="7">
    <if_sid>100100</if_sid>
    <field name="eventid">^cowrie.command.input$</field>
    <description>Honeypot : commande tapee par l'attaquant : $(input)</description>
  </rule>
</group>

active
```

Capturé le 29/10/2026 11:48:31

4.2b — Validation des règles avec wazuh-logtest

On rejoue trois événements réels (échec, connexion réussie, commande) : chacun déclenche désormais la bonne règle, avec le bon niveau. La connexion réussie atteint le **niveau 12**.

sisr@siem-01

```
sisr@siem-01:~$ wazuh-logtest < (échec, succès, commande)
> "eventid":"cowrie.login.failed"
        id: '100101'
        level: '5'
        description: 'Honeypot : echec de connexion root/123456 depuis 192.168.41.100'

> "eventid":"cowrie.login.success"
        id: '100102'
        level: '12'
        description: 'Honeypot : connexion REUSSIE sur le leurre (root depuis 192.168.41.100)'

> "eventid":"cowrie.command.input"
        id: '100103'
        level: '7'
        description: 'Honeypot : commande tapee par l'attaquant : cat /etc/passwd'
```

Capturé le 29/10/2026 11:48:31

4.3a — Serveur mail local et notifications Wazuh

Le TP autorise un webhook **ou un mail**. Choix retenu : le mail, **sans compte externe et sans rien faire sortir du labo**.

- `sudo apt install -y postfix mailutils` : serveur mail local, configuré pour n'écouter que sur la boucle locale (`inet_interfaces = loopback-only`), il n'est donc pas exposé sur le réseau
- dans `/var/ossec/etc/ossec.conf` : `email_notification yes`, `smtp_server localhost`, destinataire `sisr@localhost`
- Wazuh envoie un mail pour toute alerte de niveau **12 ou plus** (`email_alert_level`) : c'est exactement le niveau de la règle 100102 (connexion réussie sur le leurre).

Avec un compte Discord ou Teams, on utiliserait plutôt le bloc `<integration>` (nom `slack`, URL du webhook suivie de `/slack`, `rule_id 100102`).

sisr@siem-01

```
sisr@siem-01:~$ dpkg -l postfix mailutils ; postconf inet_interfaces mydestination ; grep -E 'email|smtp'
/var/ossec/etc/ossec.conf
mailutils 1:3.15-4
postfix 3.7.11-0+deb12u1
inet_interfaces = loopback-only
mydestination = localhost, siem-01

    <email_notification>yes</email_notification>
    <smtp_server>localhost</smtp_server>
    <email_from>wazuh@siem-01</email_from>
    <email_to>sisr@localhost</email_to>
    <email_maxperhour>12</email_maxperhour>
    <email_log_source>alerts.log</email_log_source>
  <alerts>
    <log_alert_level>3</log_alert_level>
    <email_alert_level>12</email_alert_level>
  </alerts>

active active
```

Capturé le 29/10/2026 11:54:20

4.3b — L'alerte reçue par mail

Test : connexion depuis Kali avec `admin123`, puis `whoami` et `exit`. Moins d'une minute après, le mail d'alerte de **niveau 12** arrive dans la boîte de `sisr` sur le SIEM, avec l'agent concerné (honeypot-01), la règle 100102 et l'événement brut (IP source, identifiant, mot de passe).

sisr@siem-01

```
sisr@siem-01:~$ mail   # puis 1 pour ouvrir le message
From wazuh@siem-01  Thu Oct 29 11:53:17 2026
From: Wazuh <wazuh@siem-01>
To: <sisr@localhost>
Date: Thu, 29 Oct 2026 11:53:17 +0100
Subject: Wazuh notification - (honeypot-01) any - Alert level 12


Wazuh Notification.
2026 Oct 29 11:53:06

Rule: 100102 fired (level 12) -> "Honeypot : connexion REUSSIE sur le leurre (root depuis 192.168.41.100)"
Portion of the log(s):

{"session":"f64b4b5f9c7f","protocol":"ssh","src_ip":"192.168.41.100","src_port":60902,"dst_ip":"172.17.0.
2","dst_port":2222,"username":"root","password":"admin123","eventid":"cowrie.login.success","sensor":"1fb7
33f121c8","uuid":"5458743a-d0ea-11f1-9b92-2aedfafa4905","timestamp":"2026-10-29T10:53:06.001720Z","messag
e":"login attempt [root/admin123] succeeded"}
session: f64b4b5f9c7f
src_ip: 192.168.41.100
src_port: 60902
dst_ip: 172.17.0.2
dst_port: 2222
username: root
password: admin123
eventid: cowrie.login.success
sensor: 1fb733f121c8
uuid: 5458743a-d0ea-11f1-9b92-2aedfafa4905
timestamp: 2026-10-29T10:53:06.001720Z
message: login attempt [root/admin123] succeeded

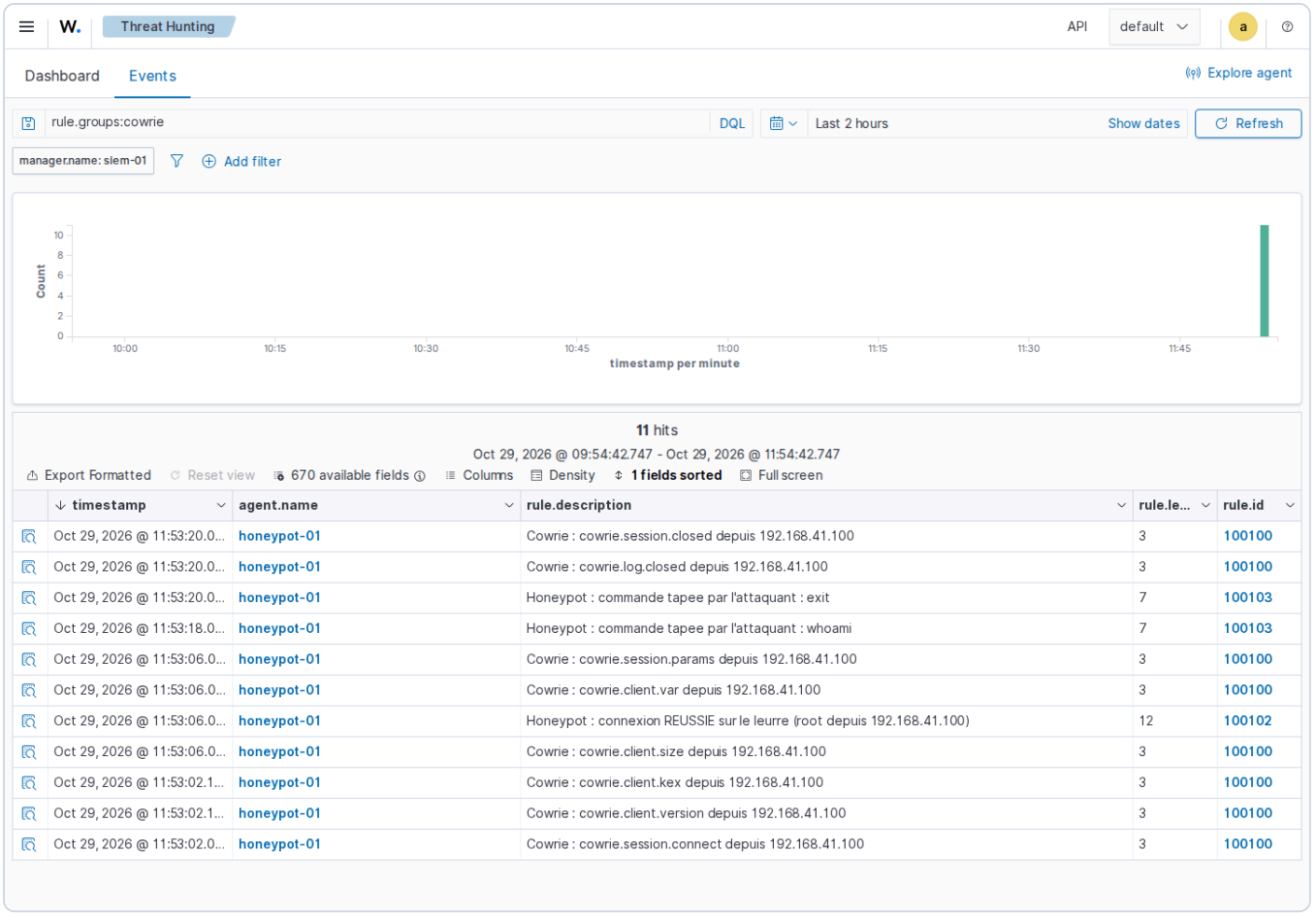



 --END OF NOTIFICATION
```

Capturé le 29/10/2026 11:54:20

4.3c — Les alertes du honeypot dans le tableau de bord Wazuh

Dans **Threat Hunting → Events**, filtre `rule.groups : cowrie` : toutes les alertes du leurre sont centralisées dans le SIEM, avec la règle, le niveau et la description. La connexion réussie ressort en **niveau 12**.



timestamp	agent.name	rule.description	rule.le...	rule.id
Oct 29, 2026 @ 11:53:20.0...	honeypot-01	Cowrie : cowrie.session.closed depuis 192.168.41.100	3	100100
Oct 29, 2026 @ 11:53:20.0...	honeypot-01	Cowrie : cowrie.log.closed depuis 192.168.41.100	3	100100
Oct 29, 2026 @ 11:53:20.0...	honeypot-01	Honeypot : commande tapee par l'attaquant : exit	7	100103
Oct 29, 2026 @ 11:53:18.0...	honeypot-01	Honeypot : commande tapee par l'attaquant : whoami	7	100103
Oct 29, 2026 @ 11:53:06.0...	honeypot-01	Cowrie : cowrie.session.params depuis 192.168.41.100	3	100100
Oct 29, 2026 @ 11:53:06.0...	honeypot-01	Cowrie : cowrie.client.var depuis 192.168.41.100	3	100100
Oct 29, 2026 @ 11:53:06.0...	honeypot-01	Honeypot : connexion REUSSIE sur le leurre (root depuis 192.168.41.100)	12	100102
Oct 29, 2026 @ 11:53:06.0...	honeypot-01	Cowrie : cowrie.client.size depuis 192.168.41.100	3	100100
Oct 29, 2026 @ 11:53:02.1...	honeypot-01	Cowrie : cowrie.client.kex depuis 192.168.41.100	3	100100
Oct 29, 2026 @ 11:53:02.1...	honeypot-01	Cowrie : cowrie.client.version depuis 192.168.41.100	3	100100
Oct 29, 2026 @ 11:53:02.0...	honeypot-01	Cowrie : cowrie.session.connect depuis 192.168.41.100	3	100100

Source : https://192.168.41.102/app/threat-hunting#/overview/?tab=general&tabView=events&_g=(time:(from:now-2h,to:now))&_a=(filters:!(),query:(language:kuery,query:'rule.groups:cowrie'))
Capturé le 29/10/2026 11:54:54

Réponses aux questions
Pourquoi le collecteur ne doit-il jamais être installé sur le leurre lui-même ?

- **Le leurre est fait pour être attaqué.** On doit partir du principe qu'il sera un jour compromis. Si le SIEM était dessus, l'attaquant pourrait **effacer ou falsifier les logs**, c'est-à-dire les preuves de son passage, et désactiver les alertes.
- En envoyant les événements **en continu vers une autre machine**, chaque action est déjà en sécurité sur le SIEM au moment où elle se produit : même si le leurre « brûle », la preuve est conservée.
- Installer le collecteur sur le leurre obligerait aussi à **ouvrir davantage le leurre** (interface web, base de données, accès administrateur), alors que le confinement vise au contraire à le réduire au strict minimum. Ce serait aussi donner à l'attaquant une vue sur notre outil de détection.
- C'est le principe de **séparation des rôles** : le capteur capte, le SIEM stocke et analyse.

Choix techniques de la partie 4

- Transport : l'**agent Wazuh** (et non Filebeat → Logstash comme dans l'énoncé), car le SIEM retenu est Wazuh, qui n'utilise pas Logstash. La logique est la même : un agent léger lit `cowrie.json` et l'envoie, chiffré, vers une autre machine.
- Alerte : par **mail local** (Postfix sur le SIEM), solution prévue par l'énoncé (« ou un mail »), qui ne nécessite aucun compte externe et ne fait rien sortir du labo. Une connexion réussie sur le leurre déclenche l'envoi en moins d'une minute.

Partie 5 — Analyser et conclure

Qui, quand, comment, jusqu'où : on transforme les logs en une histoire.

5.1a — Profil ROBOT : attaque par dictionnaire (Hydra)

Depuis Kali, attaque automatisée sur notre propre leurre avec des listes d'identifiants et de mots de passe courants :

`hydra -L users.txt -P pass.txt -s 2222 -t 4 -V ssh://192.168.41.101`

Ce qu'on observe : **26 tentatives de connexion en 1,4 seconde** (une toutes les ~50 ms), une nouvelle session pour chaque essai, 4 connexions en parallèle, un ordre mécanique (chaque identifiant de la liste essaie `root`, puis 123456…), et **aucune commande** tapée après la connexion.

sisr@honeypot-01

```
sisr@honeypot-01:~$ jq 'select(.eventid|test("login")) | [heure, session, evenement, identifiant, mot de p
asse]' cowrie.json
10:58:08.108    821d89639fc8    login.failed    root    123456
10:58:08.119    0b806b3f6e32    login.success   root    password
10:58:08.129    f342eb501adc    login.success   root    admin
10:58:08.135    59ca1f3ff2fe    login.failed    root    root
10:58:08.233    8d03a03e6b50    login.success   admin   root
10:58:08.249    6c925f24189c    login.success   admin   123456
10:58:08.326    926c180d05bb    login.success   ubuntu  root
10:58:08.366    2d10cfdfabe9    login.success   ubuntu  123456
10:58:08.413    5bec9ceb0b0b    login.success   pi      root
10:58:08.443    d6239de1c627    login.success   pi      123456
10:58:08.486    b1d5a1dbc249    login.success   oracle  root
10:58:08.510    03563e416303    login.success   oracle  123456
10:58:08.548    546be35e0479    login.success   test    root
10:58:08.572    15fa9729f0ac    login.success   test    123456
[…]

tentatives de connexion : 26
sessions ouvertes       : 27
première / dernière     : 10:58:07.728 -> 10:58:09.147
commandes tapées        : 0
```

Capturé le 29/10/2026 12:00:56

5.1b — Profil HUMAIN : session interactive

Juste après, connexion manuelle depuis Kali, en tapant les commandes à la main. La colonne 2 donne le temps écoulé depuis l'événement précédent : **4 s pour taper le mot de passe**, puis **7 à 37 s entre deux commandes**, rythme irrégulier, **une faute de frappe** (`wgte` au lieu de `wget`, enregistrée en `command.failed`), et des commandes de **reconnaissance** typiques d'un intrus qui découvre la machine (`ls -la`, `cd /tmp`, `cat /etc/shadow`, `ps aux`).

```
sisr@honeypot-01
sisr@honeypot-01:~$ jq 'select(.session=="535d6899db87") | [heure, écart, événement, commande]' cowrie.jso
n
10:58:34             session.connect depuis 192.168.41.100
10:58:38  +  4.0 s   login.success   root / admin123
10:58:49  + 11.0 s   command.input   ls -la
10:58:57  +  7.7 s   command.input   cd /tmp
10:59:05  +  7.8 s   command.input   wgte http://exemple.com/outil.sh
10:59:05  +  0.0 s   command.failed  wgte http://exemple.com/outil.sh
10:59:42  + 37.1 s   command.input   cat /etc/shadow
10:59:51  +  8.5 s   command.input   ps aux
10:59:58  +  7.9 s   command.input   exit
```

Capturé le 29/10/2026 12:00:56

5.1c — Top 5 des identifiants et des mots de passe tentés

Sur l'ensemble des journaux du TP (essais manuels et attaque automatisée).

```
sisr@honeypot-01
sisr@honeypot-01:~$ jq -r 'select(.eventid|test("cowrie.login")) | .username' cowrie.json | sort | uniq -c
| sort -rn | head -5   (idem avec .password)
== Top 5 identifiants ==
     11 root
      3 ubuntu
      3 admin
      2 user
      2 test

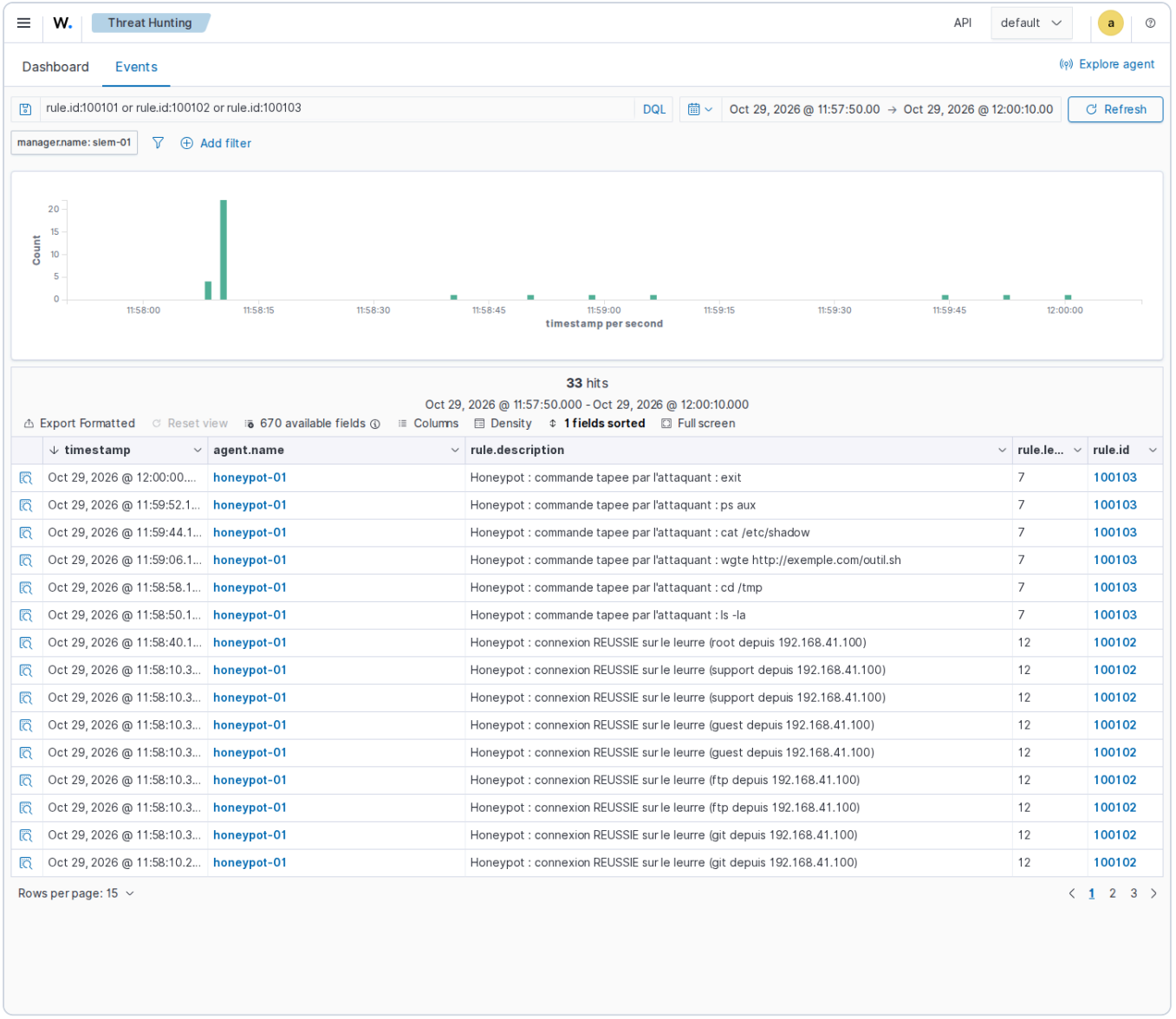
== Top 5 mots de passe ==
     13 root
     13 123456
      3 admin123
      2 admin
      1 ubuntu

total des tentatives : 35
IP sources distinctes : 192.168.41.100
```

Capturé le 29/10/2026 12:01:06

5.1d — Vue SIEM : la rafale du robot puis la session humaine

Dans le SIEM (heure locale), filtre sur les règles 100101 à 100103 entre 11:57:50 et 12:00:10 : le **pic** de l'histogramme correspond à la rafale de connexions du robot (des dizaines d'alertes de niveau 12 dans la même seconde) ; viennent ensuite les commandes de la session humaine, espacées de plusieurs secondes.



Source : https://192.168.41.102/app/threat-hunting#/overview/?tab=general&tabView=events&_g=(time:(from:'2026-10-29T10:57:50.000Z',to:'2026-10-29T11:00:10.000Z'))&_a=(filters:!(),query:(language:kuery,query:'rule.id:100101 or rule.id:100102 or rule.id:100103'))
Capturé le 29/10/2026 12:02:01

Réponses aux questions

Top 5 des identifiants et des mots de passe tentés (35 tentatives, une seule IP source : Kali)

Rang	Identifiant	Nombre	Mot de passe	Nombre
1	root	11	root	13
2	ubuntu	3	123456	13
3	admin	3	admin123	3
4	user	2	admin	2

5	test	2	ubuntu	1

Ce qui surprend : `root` et `123456` arrivent en tête alors que ce sont les mots de passe les plus connus au monde. C'est précisément pour cela que les robots les essaient en premier : ils fonctionnent encore sur des milliers de machines. On remarque aussi des comptes de service (`postgres`, `oracle`, `git`, `ftp`) et d'objets connectés (`pi` du Raspberry Pi) : les attaquants visent les comptes installés par défaut, que les administrateurs oublient de sécuriser.

Verdict robot / humain

Indice	Session « Hydra »	Session interactive
Rythme	26 tentatives en 1,4 s (une toutes les ~50 ms), régulier	4 s pour le mot de passe, puis 7 à 37 s entre deux commandes, irrégulier
Sessions	27 sessions, 4 en parallèle	1 seule session
Ordre	mécanique : chaque identifiant essaie la même liste dans le même ordre	commandes choisies au fil de l'exploration
Erreurs	aucune	faute de frappe `wgte` (`command.failed`)
Après connexion	aucune commande : le robot ne fait que tester les identifiants	reconnaissance (`ls -la`, `cd /tmp`, `cat /etc/shadow`, `ps aux`)

Verdict : la première activité est un **robot** (rafale rapide, régulière, parallèle, répétitive, sans exploitation). La seconde est un **humain** (rythme lent et irrégulier, hésitations, faute de frappe, commandes de découverte, tentative de téléchargement d'un outil dans `/tmp`).
Remarque : les 28 alertes de niveau 12 de la rafale n'ont produit que 3 mails, car Wazuh regroupe les alertes et plafonne les envois (`email_maxperhour` = 12). Cela évite de noyer l'analyste, mais montre qu'il faut aussi regarder le tableau de bord.

Trois recommandations concrètes pour une PME

- **Supprimer l'authentification SSH par mot de passe** : connexion par **clé** uniquement (`PasswordAuthentication no`), `PermitRootLogin no`, et des mots de passe robustes partout ailleurs. Les mots de passe par défaut des comptes de service et des équipements (`root`, `admin`, `pi`, `oracle`, `postgres`) doivent être changés dès l'installation. Toutes les tentatives observées ici visaient ce type de compte.
- **Ne pas exposer SSH directement sur Internet** : passer par un VPN ou filtrer par IP sur le pare-feu, et bloquer automatiquement les IP qui enchaînent les échecs (**fail2ban**, ou réponse active Wazuh). Une rafale comme celle du robot serait coupée au bout de quelques essais.
- **Centraliser les journaux et alerter** : envoyer les logs de chaque serveur vers un SIEM (Wazuh est gratuit), avec des alertes sur les connexions réussies anormales et les rafales d'échecs. Si un attaquant compromet un serveur, les preuves sont déjà à l'abri, comme dans ce TP. On peut aussi déployer un petit honeypot interne : sur le réseau d'une PME, personne ne devrait jamais s'y connecter, donc **toute connexion** est une alerte fiable de présence d'un intrus.

Bonus constaté pendant le TP : ne jamais réutiliser un vrai mot de passe (le mot de passe réel du lab, tapé par erreur, a été stocké en clair dans les journaux du leurre). Et confiner tout système exposé : sortie fermée par défaut, comme en partie 3.

Synthèse

6.1 — Bilan et difficultés rencontrées

Bilan : la chaîne complète fonctionne : le leurre attire et enregistre, il est confiné (sortie refusée par défaut, preuves à l'appui), ses journaux partent en continu vers un SIEM situé sur une autre machine, et une connexion réussie déclenche une alerte en moins d'une minute.

Difficultés rencontrées et solutions

Problème	Cause	Solution
Script d'installation Wazuh illisible	l'URL `/4.x/` renvoie une erreur 403, la page d'erreur a été enregistrée à la place du script	URL versionnée `/4.14/wazuh-install.sh`
`gpg : commande introuvable`	image Debian « cloud » minimale	`sudo apt install -y gnupg`
Agent « Unknown », indexer en erreur	disque du SIEM (20 Go) saturé par la base de vulnérabilités	disque agrandi à 60 Go à chaud, index déverrouillés
Copier-coller impossible dans la console des VM	console texte sans presse-papiers	travail en SSH depuis le poste
Filebeat → Logstash (énoncé) inadapté	Wazuh n'utilise pas Logstash	transport par l'agent Wazuh
Pas de compte Discord / Teams	—	alerte par mail local (option prévue par l'énoncé)

Ce que j'en retiens : un honeypot est autant une affaire de **confinement** que de piégeage. Une alerte n'a de valeur que si elle est **rare et fiable** : sur un leurre, une connexion réussie l'est toujours. Enfin, dimensionner correctement le SIEM est indispensable, sinon c'est l'outil de surveillance lui-même qui tombe.

Capturé le 29/10/2026 12:02:47